David Mohaisen, Professor
University of Central Florida
Department of Computer Science
Cybersecurity and Privacy Cluster

https://cs.ucf.edu/~mohaisen/

Recurrent Language Models

① Recurrent Foundations
② ANN vs RNN
③ Training RNNs
④ Vanishing Gradients
⑤ LSTM and GRU
⑥ LM Evaluation

Fixed-Window Language Model

~~as the proctor started the clock~~ the students opened their _____

discard

fixed window

Fixed-Window Language Model

output distribution

$$\hat{\boldsymbol{y}} = \text{softmax}(\boldsymbol{U}\boldsymbol{h} + \boldsymbol{b}_2) \in \mathbb{R}^{|V|}$$

hidden layer

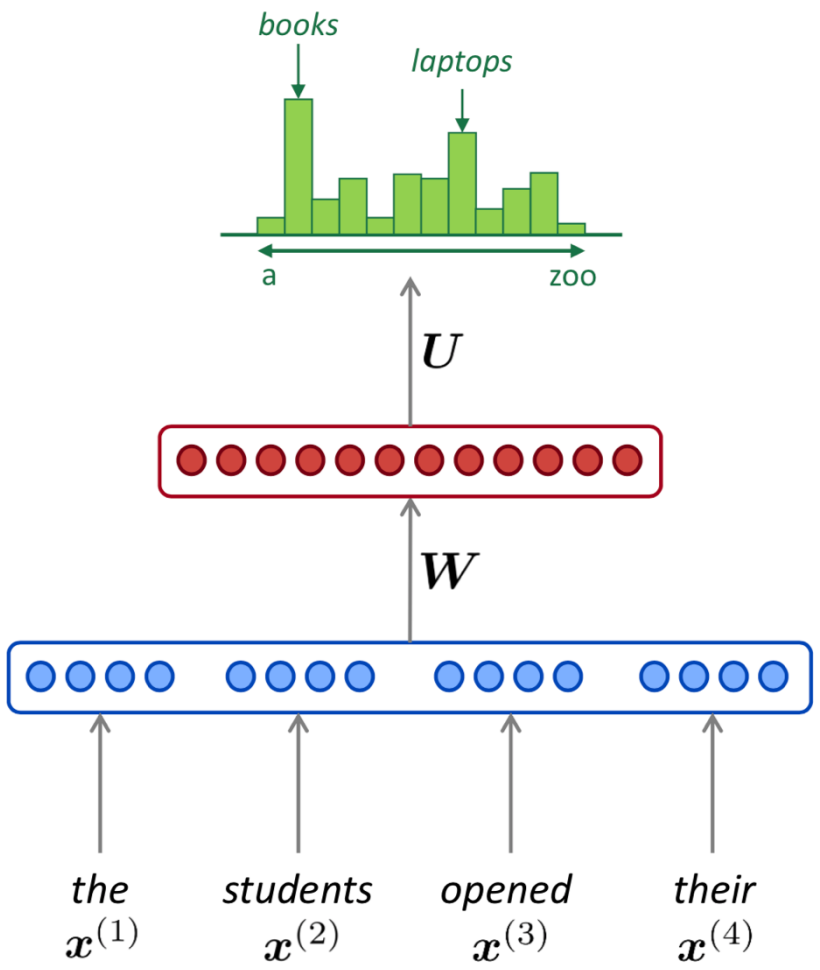
$$\boldsymbol{h} = f(\boldsymbol{W}\boldsymbol{e} + \boldsymbol{b}_1)$$

concatenated word embeddings

$$\boldsymbol{e} = [\boldsymbol{e}^{(1)}; \boldsymbol{e}^{(2)}; \boldsymbol{e}^{(3)}; \boldsymbol{e}^{(4)}]$$

words / one-hot vectors

$$\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \boldsymbol{x}^{(3)}, \boldsymbol{x}^{(4)}$$



Fixed-Window Language Model

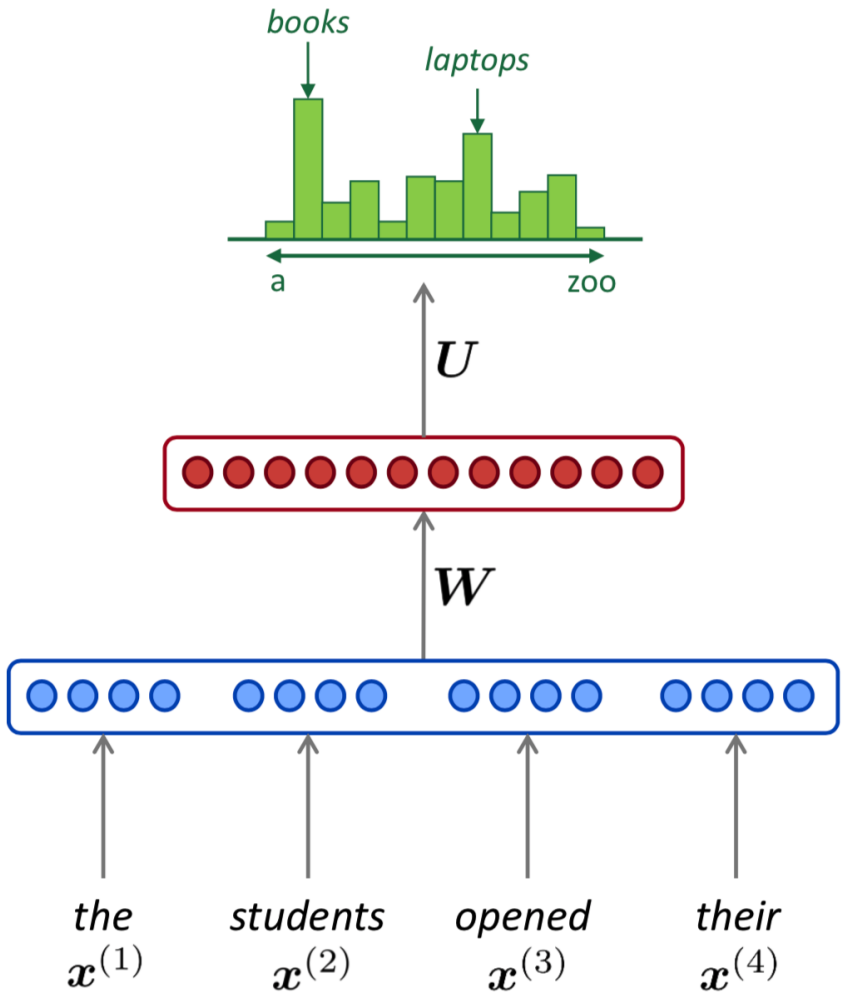
Improvements over *n*-gram LM:

- No sparsity problem
- Don't need to store all observed *n*-grams

Remaining **problems**:

- Fixed window is too small
- Enlarging window enlarges $\boldsymbol{W}$
- Window can never be large enough!
- $\boldsymbol{x}^{(1)}$ and $\boldsymbol{x}^{(2)}$ are multiplied by completely different weights in $\boldsymbol{W}$. No symmetry in how the inputs are processed.

We need a neural architecture that can process *any length input*



ANN vs. RNN – Key Differences

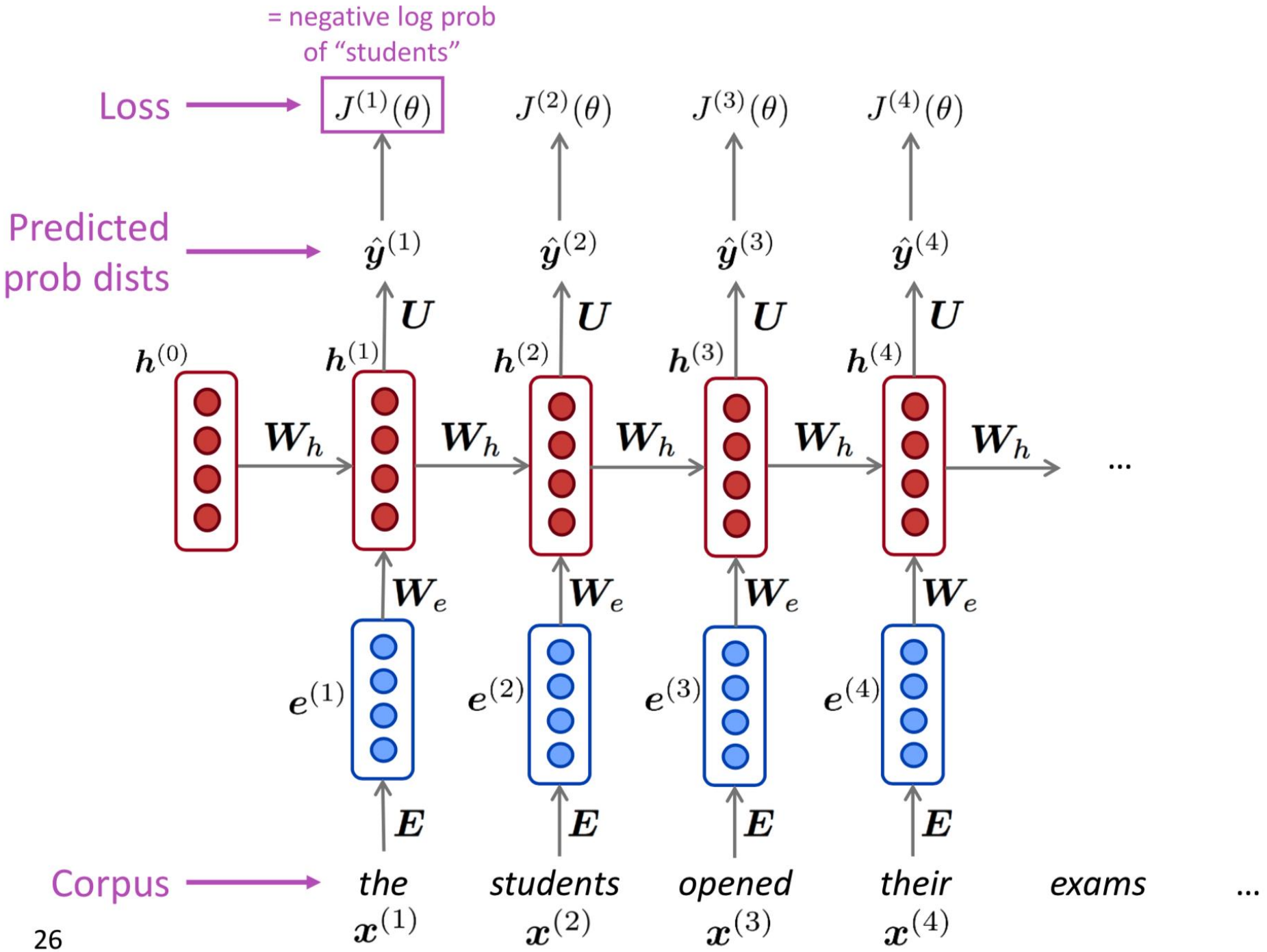
- **1. Architecture & Information Flow**
  - **ANN (Artificial Neural Network)**:
    - **Feedforward** network (no loops). Each neuron processes input **independently**.
    - Best for tasks where inputs are **not sequential** (e.g., image classification).
  - **RNN (Recurrent Neural Network)**:
    - **Recurrent connections** allow memory retention. Outputs depend on **current + previous inputs**.
    - Best for **time-dependent** data (e.g., speech, text, time-series).
- **2. Memory & Sequence Handling**
  - **ANN**: No memory, handles each input separately.
  - **RNN**: Maintains memory via hidden states, enabling **context-aware learning**.
- **3. Training Complexity**
  - **ANN**: Easier to train, standard backpropagation.
  - **RNN**: Harder to train, suffers from **vanishing/exploding gradients**, requires **BPTT (Backpropagation Through Time)**.

ANN vs. RNN – Key Differences

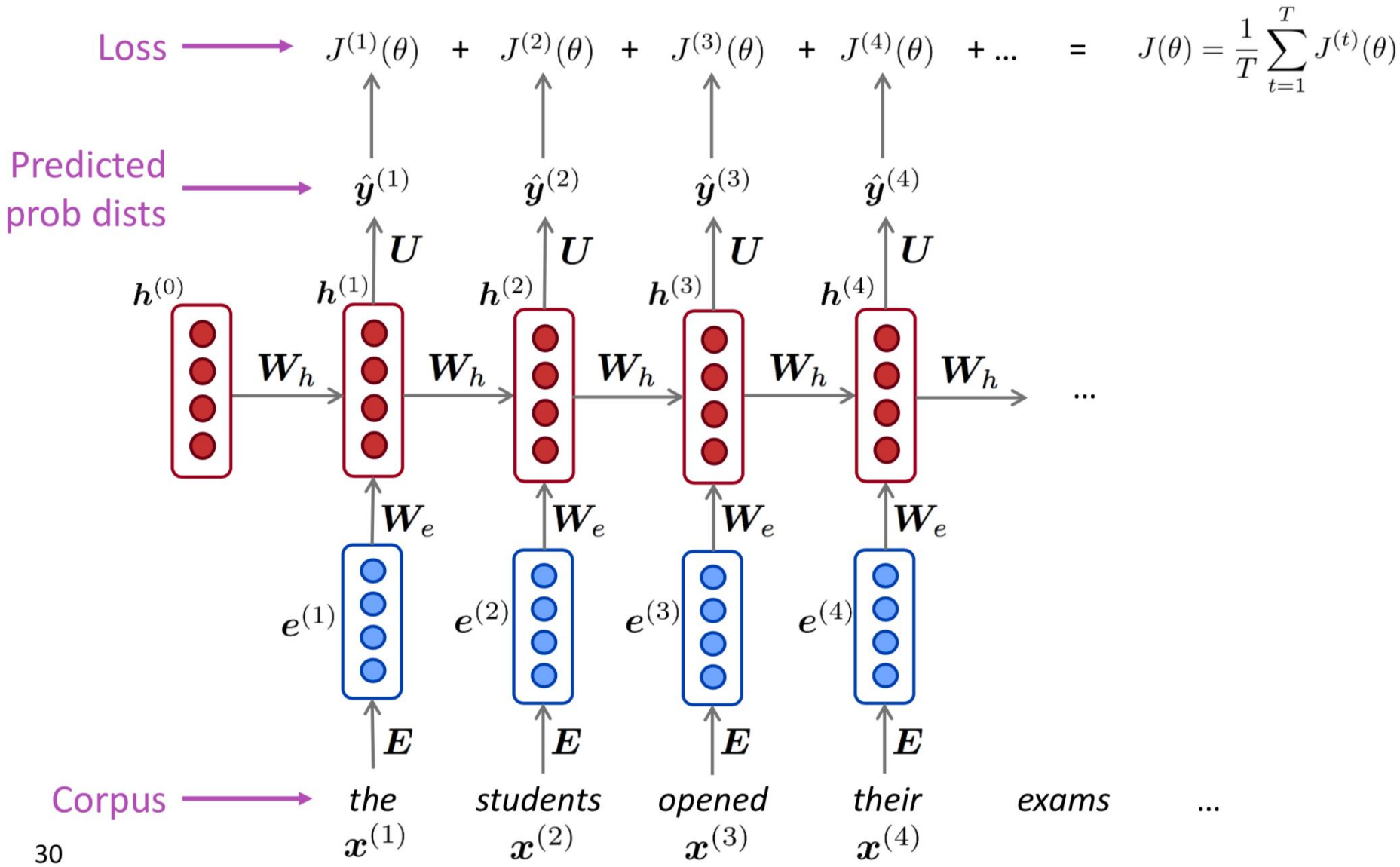
- Summary:
  - **Use ANN** for static data (e.g., images, structured data).
  - **Use RNN** for sequential data (e.g., text, speech, time-series).
  - For long sequences, **LSTM/GRU** improves RNN performance.

Feature	ANN	RNN
Independent data	✅	❌
Sequential data	❌	✅
Memory	No	Yes
Best for	Image classification, fraud detection	Speech recognition, text generation
Training	Easier	Harder (needs BPTT)

RNN


= negative log prob
of "students"
Loss
$J^{(1)}(\theta)$
$J^{(2)}(\theta)$
$J^{(3)}(\theta)$
$J^{(4)}(\theta)$
Predicted
prob dists
$\hat{\boldsymbol{y}}^{(1)}$
$\hat{\boldsymbol{y}}^{(2)}$
$\hat{\boldsymbol{y}}^{(3)}$
$\hat{\boldsymbol{y}}^{(4)}$
$\boldsymbol{U}$
$\boldsymbol{h}^{(0)}$
$\boldsymbol{h}^{(1)}$
$\boldsymbol{h}^{(2)}$
$\boldsymbol{h}^{(3)}$
$\boldsymbol{h}^{(4)}$
$\boldsymbol{W}_h$
...
$\boldsymbol{W}_e$
$\boldsymbol{e}^{(1)}$
$\boldsymbol{e}^{(2)}$
$\boldsymbol{e}^{(3)}$
$\boldsymbol{e}^{(4)}$
$\boldsymbol{E}$
Corpus
the
students
opened
their
exams
...
$\boldsymbol{x}^{(1)}$
$\boldsymbol{x}^{(2)}$
$\boldsymbol{x}^{(3)}$
$\boldsymbol{x}^{(4)}$

RNN


Loss
$J^{(1)}(\theta)$ + $J^{(2)}(\theta)$ + $J^{(3)}(\theta)$ + $J^{(4)}(\theta)$ + ... = $J(\theta) = \frac{1}{T}\sum_{t=1}^{T} J^{(t)}(\theta)$
Predicted prob dists
$\hat{\boldsymbol{y}}^{(1)}$ $\hat{\boldsymbol{y}}^{(2)}$ $\hat{\boldsymbol{y}}^{(3)}$ $\hat{\boldsymbol{y}}^{(4)}$
$\boldsymbol{U}$
$\boldsymbol{h}^{(0)}$ $\boldsymbol{h}^{(1)}$ $\boldsymbol{h}^{(2)}$ $\boldsymbol{h}^{(3)}$ $\boldsymbol{h}^{(4)}$
$\boldsymbol{W}_h$
...
$\boldsymbol{W}_e$
$\boldsymbol{e}^{(1)}$ $\boldsymbol{e}^{(2)}$ $\boldsymbol{e}^{(3)}$ $\boldsymbol{e}^{(4)}$
$\boldsymbol{E}$
Corpus
the students opened their exams ...
$\boldsymbol{x}^{(1)}$ $\boldsymbol{x}^{(2)}$ $\boldsymbol{x}^{(3)}$ $\boldsymbol{x}^{(4)}$

30

Training RNN-based LM

- Computing the loss and gradients across entire corpus of $x^{(1)}, \ldots, x^{(T)}$ is too expensive.

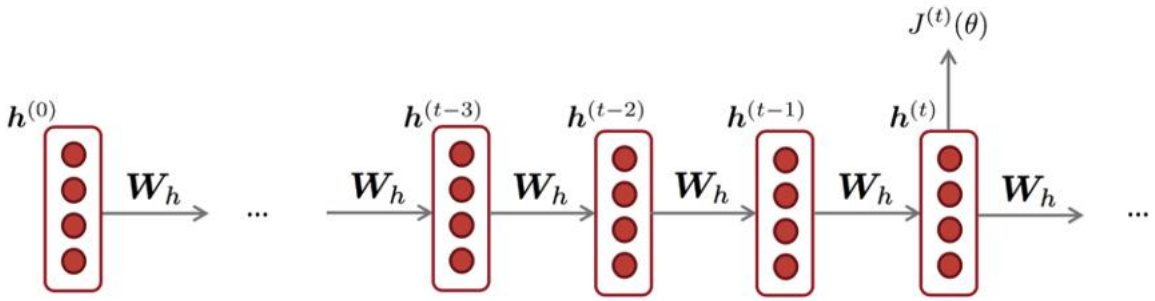
$$J(\theta) = \frac{1}{T} \sum_{t=1}^{T} J^{(t)}(\theta)$$

- In practice, consider as $x^{(1)}, \ldots, x^{(T)}$ a sentence (or a document).
- Recall: stochastic gradient descent allows us to compute loss and gradients for small chunk of data, and update.
- Compute the loss $J(\theta)$ for a sentence (or a batch of sentences), compute the gradients and update weights, then repeat until convergence.

Backpropagation for RNN

- Question: what's the derivative of $J^{(t)}(\theta)$ w.r.t. repeated weight matrix W_h?

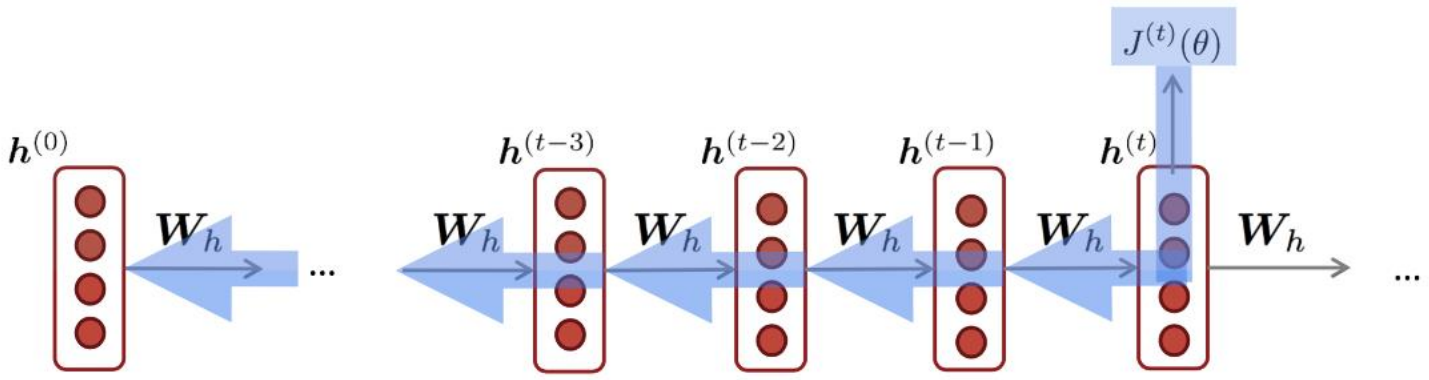
- Answer:



$$\frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}} = \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}}\bigg|_{(i)}$$

The gradient w.r.t. a repeated weight is the sum of the gradient w.r.t. each time it appears.

Backpropagation for RNN

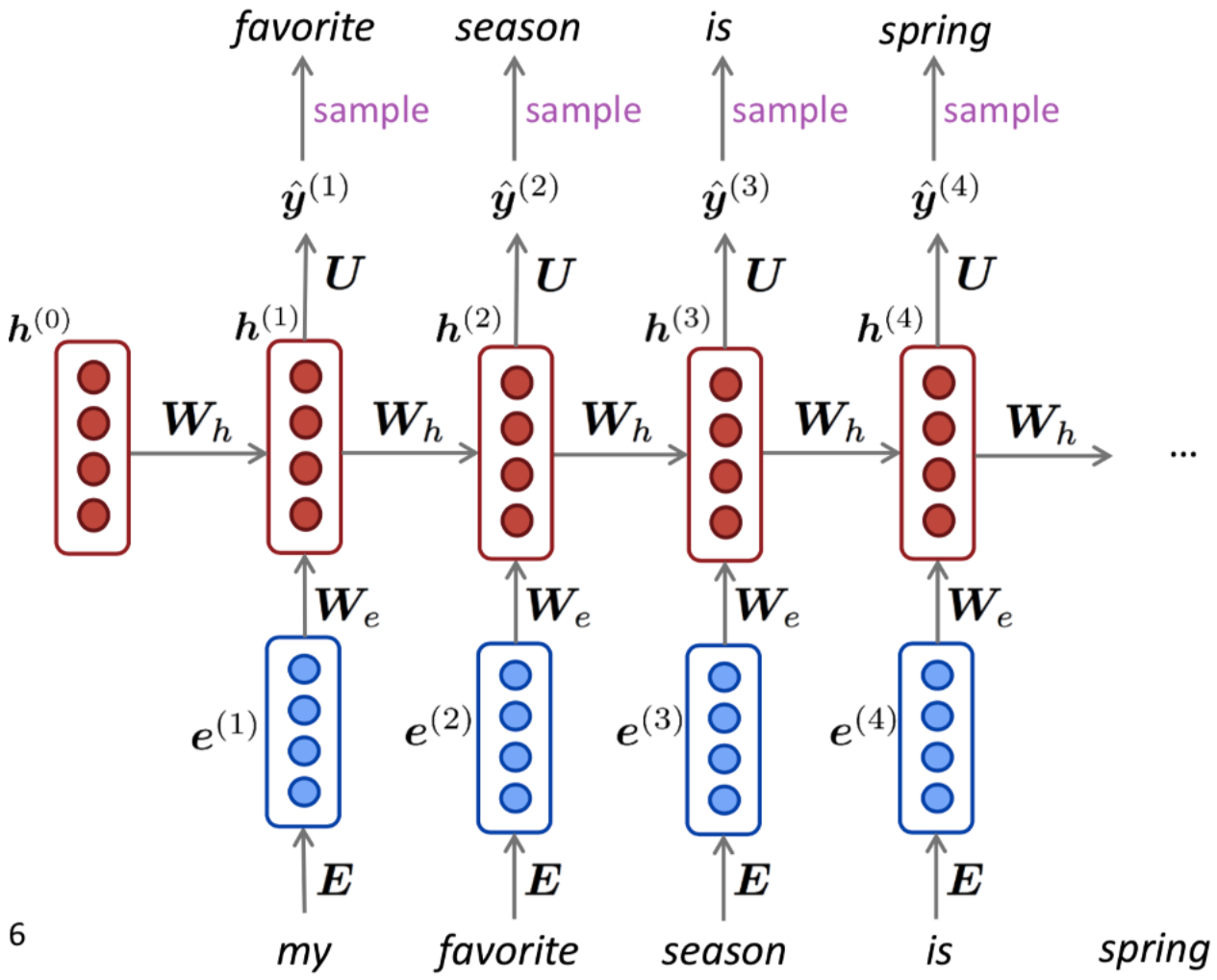


$$\frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}} = \boxed{\sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}}\bigg|_{(i)}}$$

- Answer: backpropagate over timesteps i=1,…,0 summing gradients as you go. This algorithm is called "backpropagation through time".

- Just like n-gram LM, you can use RNN LM to generate text by repeated sampling. Sampled output is next step's input.

  - Start with input.
  - Feed it in embedder
  - Feed it into RNN
  - Score the output
  - Decode the output
  - Sample next word.



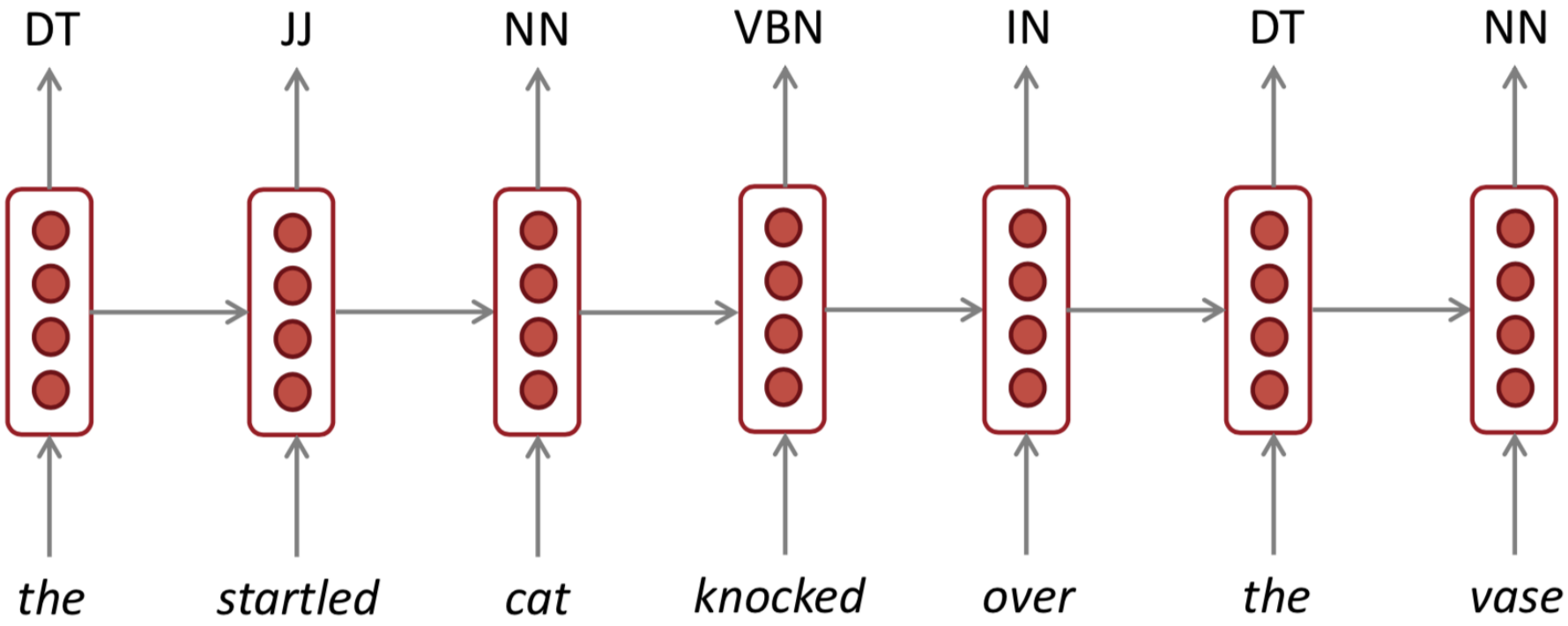
RNNs for Natural Language Generation

- **Why Use RNNs for Text Generation?**
  - **Handles sequential dependencies** in text.
  - Captures **context and structure** over time.
  - Enables **predictive text, chatbots, story generation, and machine translation**.
- **How RNNs Generate Text**
  - **Input:** Start with a sequence of words/characters.
  - **Processing:** Hidden states retain context from previous steps.
  - **Output:** Predicts the next word/character step-by-step.
  - **Recurrent Loop:** Generated output is fed back as input for the next step.
- **Challenges in Text Generation**
  - **Vanishing gradients** in long sequences.
  - **Loss of context** in standard RNNs.
  - **Repetitive or nonsensical output** without proper constraints.
  - **Solution:** Use **LSTMs or GRUs** for better long-term dependencies.

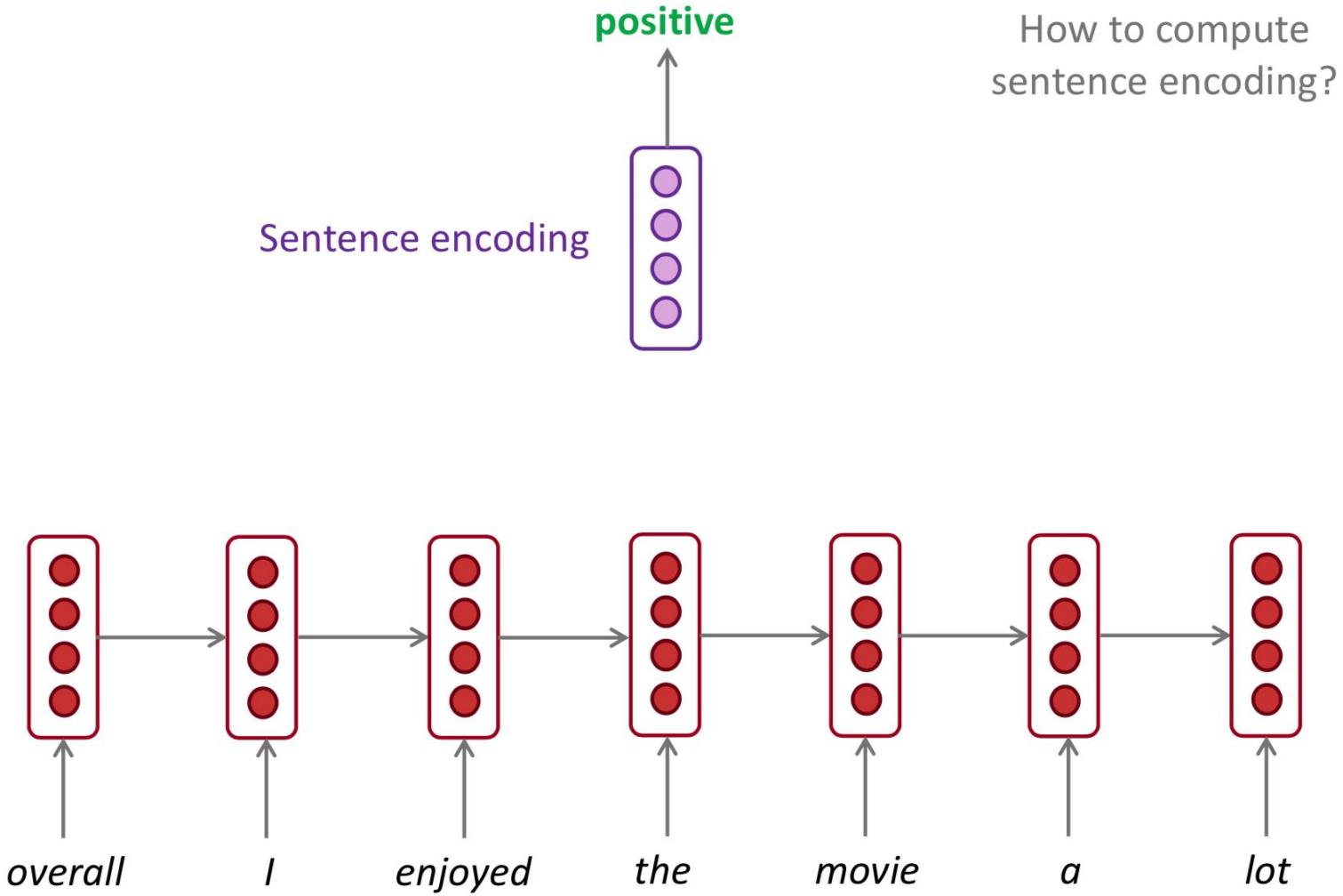
RNN-based Text Generation Techniques

- **Approaches to Generating Text with RNNs**
  - **Character-level RNNs:** Generates text letter by letter (e.g., Shakespeare-style text generation).
  - **Word-level RNNs:** Generates sentences word by word (more coherent, used in chatbots).
  - **Seq2Seq models:** Used for translation, summarization, and dialogue generation.
- **Training Process**
  - Train on large corpora of text (books, articles, dialogues).
  - Use **next-word prediction** to fine-tune the model.
  - Apply **beam search** or **temperature-based sampling** for creative outputs.
- **Improvements with Advanced Architectures**
  - **LSTMs/GRUs:** Solve vanishing gradient issues.
  - **Transformers (e.g., GPT):** More powerful, context-aware, and parallelizable than RNNs.
  - **Hybrid Models:** Combine RNNs with attention mechanisms for better fluency.

RNN Applications: POS



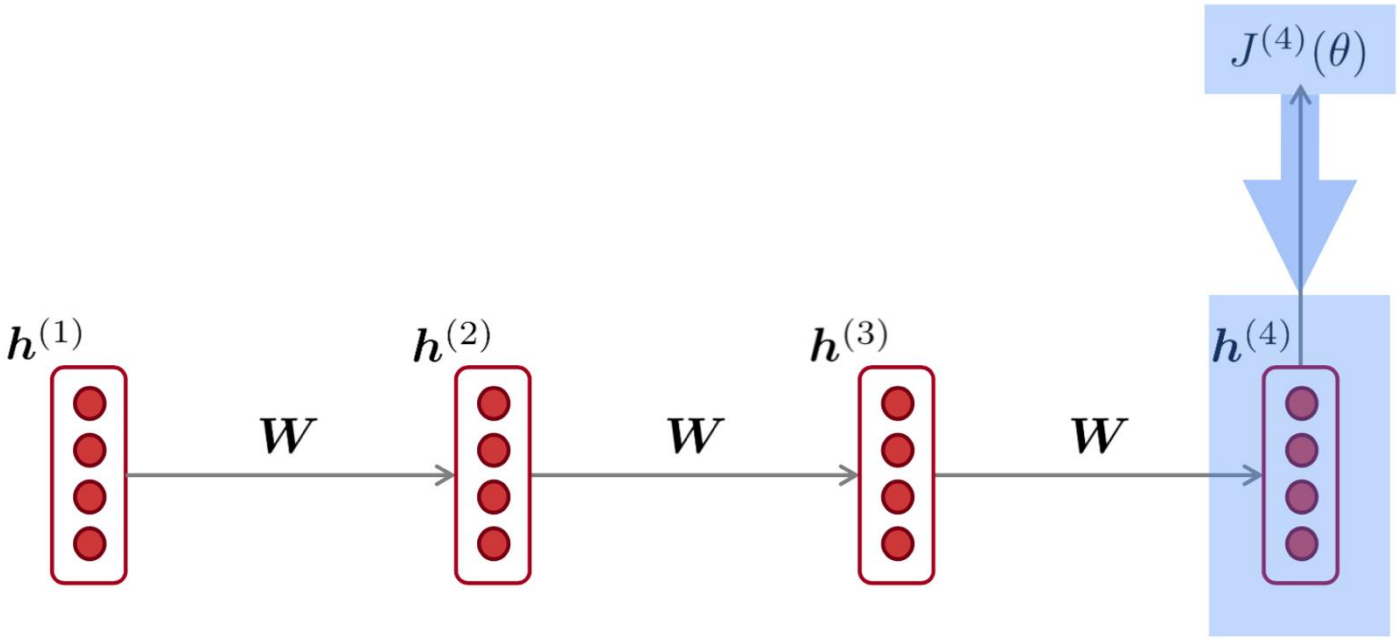
RNN Applications: Sentiment



The Vanishing Gradient Problem in RNNs

- **What is the Vanishing Gradient Problem?**
  - Occurs when training deep RNNs using **Backpropagation Through Time (BPTT)**.
  - Gradients shrink exponentially as they propagate backward through time.
  - **Result:** Early time steps contribute little to learning, making long-term dependencies difficult to capture.
- **Why Does It Happen?**
  - RNNs use **repeated weight multiplications** at each time step.
  - If weights are **<1**, gradients become **smaller** (vanishing gradient).
  - If weights are **>1**, gradients become **larger** (exploding gradient).

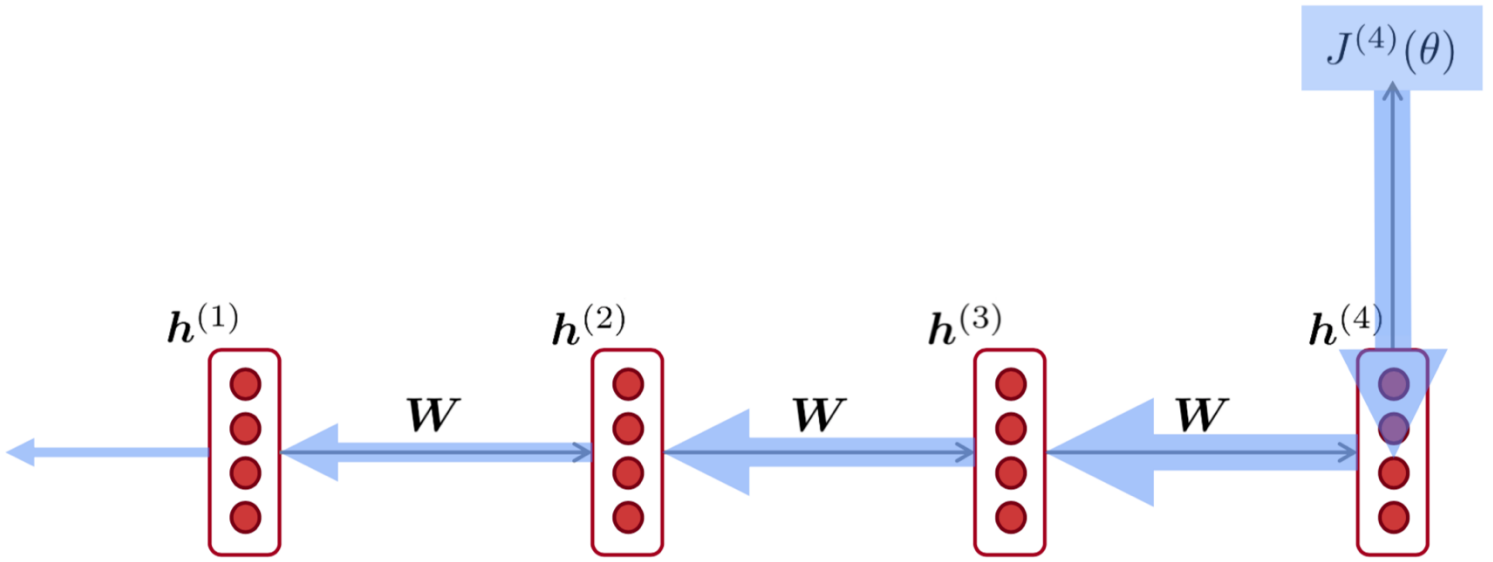
Vanishing Gradient Intuition



$$\frac{\partial J^{(4)}}{\partial \boldsymbol{h}^{(1)}} = \frac{\partial \boldsymbol{h}^{(2)}}{\partial \boldsymbol{h}^{(1)}} \times \frac{\partial \boldsymbol{h}^{(3)}}{\partial \boldsymbol{h}^{(2)}} \times \frac{\partial \boldsymbol{h}^{(4)}}{\partial \boldsymbol{h}^{(3)}} \times \frac{\partial J^{(4)}}{\partial \boldsymbol{h}^{(4)}}$$

chain rule!

Vanishing Gradient Intuition



$$\frac{\partial J^{(4)}}{\partial \boldsymbol{h}^{(1)}} = \boxed{\frac{\partial \boldsymbol{h}^{(2)}}{\partial \boldsymbol{h}^{(1)}}} \times \boxed{\frac{\partial \boldsymbol{h}^{(3)}}{\partial \boldsymbol{h}^{(2)}}} \times \boxed{\frac{\partial \boldsymbol{h}^{(4)}}{\partial \boldsymbol{h}^{(3)}}} \times \frac{\partial J^{(4)}}{\partial \boldsymbol{h}^{(4)}}$$

What happens if these are small?

<u>Vanishing gradient problem:</u>
When these are small, the gradient signal gets smaller and smaller as it backpropagates further

The Vanishing Gradient Problem in RNNs

- **Effects of Vanishing Gradients**
  - **Loss of long-term dependencies** → Model forgets earlier context.
  - **Slow or ineffective learning** in deep RNNs.
  - **Bias toward recent inputs** instead of full sequence understanding.
- **Solutions & Mitigations**
  - **LSTMs & GRUs**: Use gating mechanisms to retain long-term dependencies.
  - **Gradient Clipping**: Prevents exploding gradients.
  - **Residual Connections**: Used in modern architectures to ease training.
  - **Batch Normalization**: Helps stabilize activations.

The Math of The Vanishing Gradient

- Recall: $$\boldsymbol{h}^{(t)} = \sigma\left(\boldsymbol{W}_h \boldsymbol{h}^{(t-1)} + \boldsymbol{W}_x \boldsymbol{x}^{(t)} + \boldsymbol{b}_1\right)$$

- Therefore: $$\frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(t-1)}} = \operatorname{diag}\left(\sigma'\left(\boldsymbol{W}_h \boldsymbol{h}^{(t-1)} + \boldsymbol{W}_x \boldsymbol{x}^{(t)} + \boldsymbol{b}_1\right)\right) \boldsymbol{W}_h$$ (chain rule)

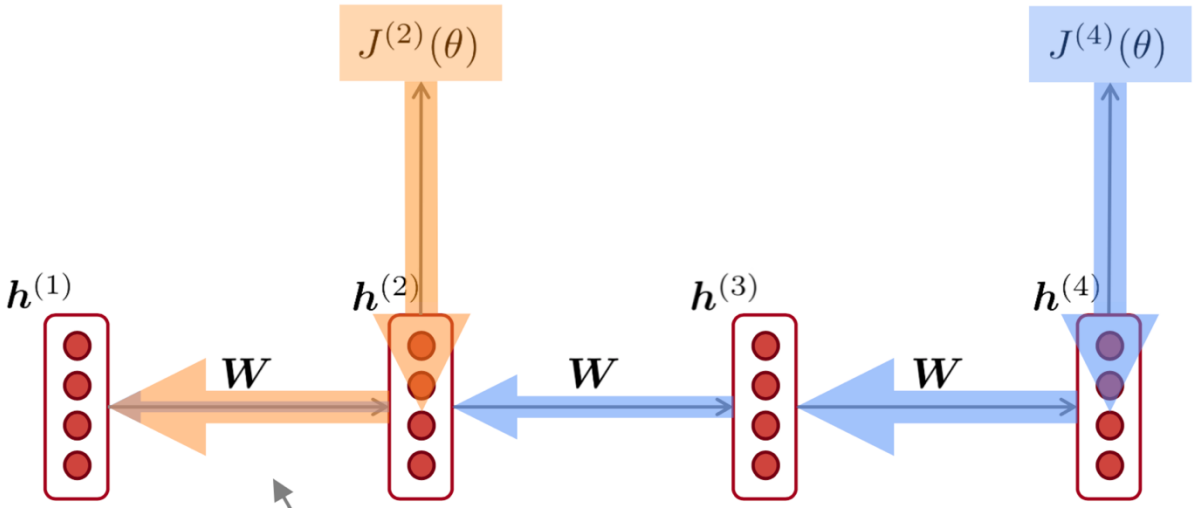
- Consider the gradient of the loss $J^{(i)}(\theta)$ on step *i*, with respect to the hidden state $\boldsymbol{h}^{(j)}$ on some previous step *j*.

$$\frac{\partial J^{(i)}(\theta)}{\partial \boldsymbol{h}^{(j)}} = \frac{\partial J^{(i)}(\theta)}{\partial \boldsymbol{h}^{(i)}} \prod_{j<t\leq i} \frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(t-1)}}$$ (chain rule)

$$= \frac{\partial J^{(i)}(\theta)}{\partial \boldsymbol{h}^{(i)}} \boxed{\boldsymbol{W}_h^{(i-j)}} \prod_{j<t\leq i} \operatorname{diag}\left(\sigma'\left(\boldsymbol{W}_h \boldsymbol{h}^{(t-1)} + \boldsymbol{W}_x \boldsymbol{x}^{(t)} + \boldsymbol{b}_1\right)\right)$$ (value of $\frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(t-1)}}$)

If W_h is small, then this term gets vanishingly small as *i* and *j* get further apart

The Actual Problem



Gradient signal from faraway is lost because it's much smaller than gradient signal from close-by.

So model weights are only updated only with respect to near effects, not long-term effects.

Recap of The Vanishing Gradient

- Gradient can be viewed as a measure of the effect of the past on the future (thus the error we see; and we try to accommodate for through the backpropagation).
- If the gradient becomes vanishingly small over longer distances (step t to step $t+n$) then we cannot tell whether
  - There is no dependency between step t and $t+n$ in the data
  - We have wrong parameters to capture the true dependency between t and $t+n$

Effect of The Vanishing Gradient

- **LM task:** *When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her* _____
- To learn from this training example, the RNN-LM needs to model the dependency between *"tickets"* on the 7th step and the target word *"tickets"* at the end.
- But if gradient is small, the model can't learn this dependency
  - So the model is unable to predict similar long-distance dependencies at test time

Effect of The Vanishing Gradient

- **LM task:** The writer of the books ___ [is, are]
- Correct answer: The writer of the book is planning a sequel.

- Syntactic recency: the writer of the books is (correct)
- Sequential recency: the writer of the books are (incorrect)

- Due to the vanishing gradient, RNN-LMs are better at learning from sequential recency than syntactic recency, so they make this type of errors more often than you'd like.

The Exploding Gradient

- If the gradient becomes too big, then the SGD update step becomes too big.

$$\theta^{new} = \theta^{old} - \overbrace{\alpha}^{\text{learning rate}} \underbrace{\nabla_\theta J(\theta)}_{\text{gradient}}$$

- **This can cause bad updates:** we take too large a step and reach a bad parameter configuration with large loss)
- In the worst case, this will result in Inf or NaN in your network (then you have to restart the training from an earlier checkpoint or from the start if you are not checkpointing.

The Exploding Gradient: Solution

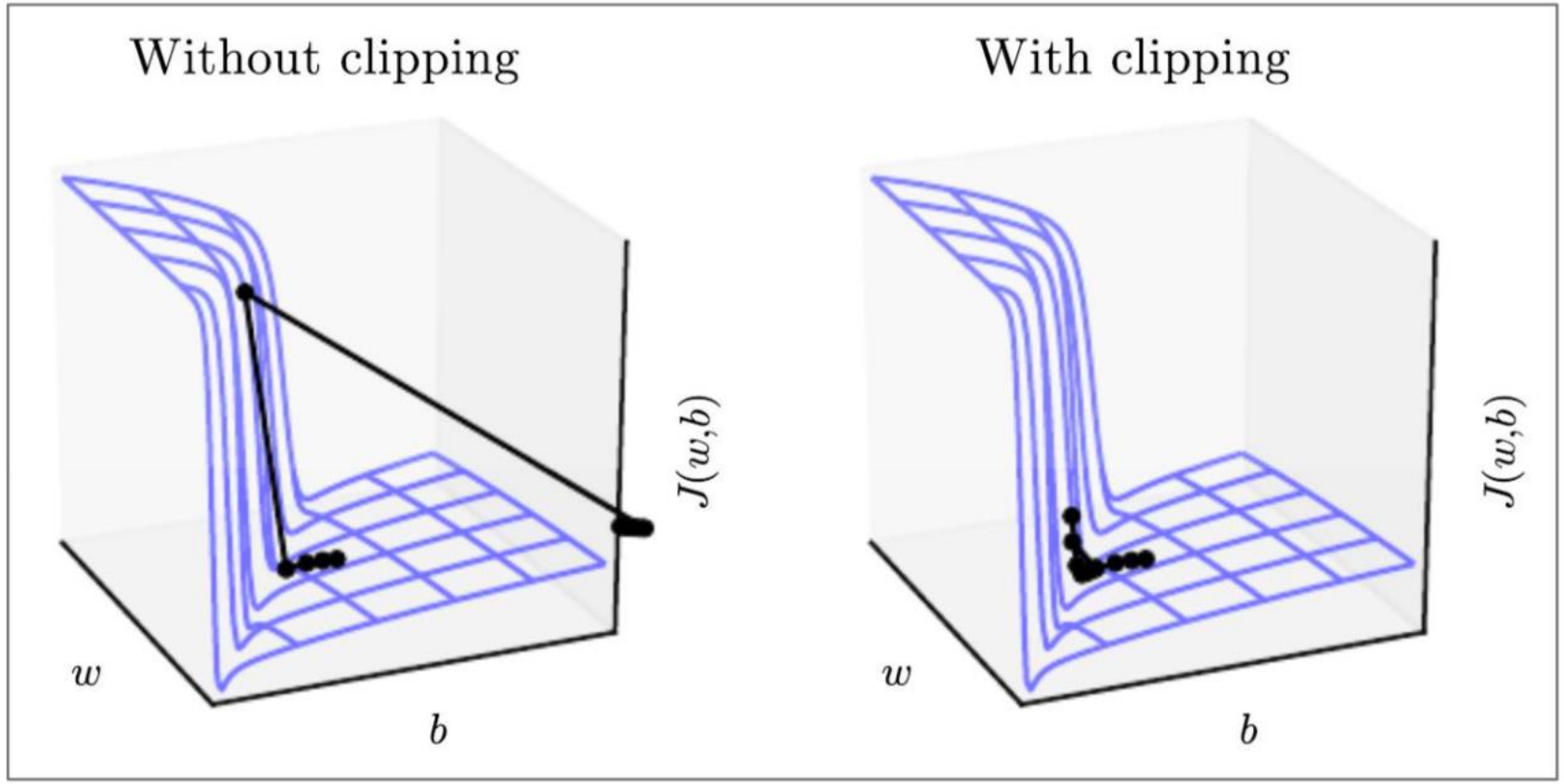
- Gradient Clipping: If the gradient is greater than a threshold, clip it (scale it down) before applying SGD updates – gradient clipping

Algorithm 1 Pseudo-code for norm clipping

$\hat{\mathbf{g}} \leftarrow \frac{\partial \mathcal{E}}{\partial \theta}$
if $\|\hat{\mathbf{g}}\| \geq threshold$ **then**
  $\hat{\mathbf{g}} \leftarrow \frac{threshold}{\|\hat{\mathbf{g}}\|}\hat{\mathbf{g}}$
end if

- Intuition: take a step in the same direction but smaller.

The Exploding Gradient: Visually



Vanishing Gradient Solution?

- The main problem is that *it's too difficult for the RNN to learn to preserve information over many timesteps.*
- In a vanilla RNN, the hidden state is constantly being rewritten

$$\boldsymbol{h}^{(t)} = \sigma\left(\boldsymbol{W}_h \boldsymbol{h}^{(t-1)} + \boldsymbol{W}_x \boldsymbol{x}^{(t)} + \boldsymbol{b}\right)$$

- How about a RNN with separate memory?

Long Short-Term Memory (LSTM)

- A type of RNN proposed by Hochreiter and Schmidhuber in 1997 as a solution to the vanishing gradients problem.
-
- On step t, there is a hidden state $\boldsymbol{h}^{(t)}$ and a cell $\boldsymbol{c}^{(t)}$
  - Both are vectors of length n
  - The cell stores long-term information
  - The LSTM can erase, write, or read info from the cell.

Long Short-Term Memory (LSTM)

- A type of RNN proposed by Hochreiter and Schmidhuber in 1997 as a solution to the vanishing gradients problem.

- The selection of which info to erase/write/read is controlled by three corresponding gates
  - The gates are also vectors of length n
  - On each step, each element of gate can be 0 or 1 (close/open) or somewhere in between).
  - The gates are dynamic: the value is computed based on current context.

Long Short-Term Memory (LSTM)

We have a sequence of inputs $\boldsymbol{x}^{(t)}$, and we will compute a sequence of hidden states $\boldsymbol{h}^{(t)}$ and cell states $\boldsymbol{c}^{(t)}$. On timestep t:

Sigmoid function: all gate values are between 0 and 1

Forget gate: controls what is kept vs forgotten, from previous cell state

$$\boldsymbol{f}^{(t)} = \boxed{\sigma}\left(\boldsymbol{W}_f \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_f \boldsymbol{x}^{(t)} + \boldsymbol{b}_f\right)$$

Input gate: controls what parts of the new cell content are written to cell

$$\boldsymbol{i}^{(t)} = \boxed{\sigma}\left(\boldsymbol{W}_i \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_i \boldsymbol{x}^{(t)} + \boldsymbol{b}_i\right)$$

Output gate: controls what parts of cell are output to hidden state

$$\boldsymbol{o}^{(t)} = \boxed{\sigma}\left(\boldsymbol{W}_o \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_o \boldsymbol{x}^{(t)} + \boldsymbol{b}_o\right)$$

New cell content: this is the new content to be written to the cell

$$\tilde{\boldsymbol{c}}^{(t)} = \tanh\left(\boldsymbol{W}_c \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_c \boldsymbol{x}^{(t)} + \boldsymbol{b}_c\right)$$

Cell state: erase ("forget") some content from last cell state, and write ("input") some new cell content

$$\boldsymbol{c}^{(t)} = \boldsymbol{f}^{(t)} \circ \boldsymbol{c}^{(t-1)} + \boldsymbol{i}^{(t)} \circ \tilde{\boldsymbol{c}}^{(t)}$$

Hidden state: read ("output") some content from the cell

$$\boldsymbol{h}^{(t)} = \boldsymbol{o}^{(t)} \circ \tanh \boldsymbol{c}^{(t)}$$

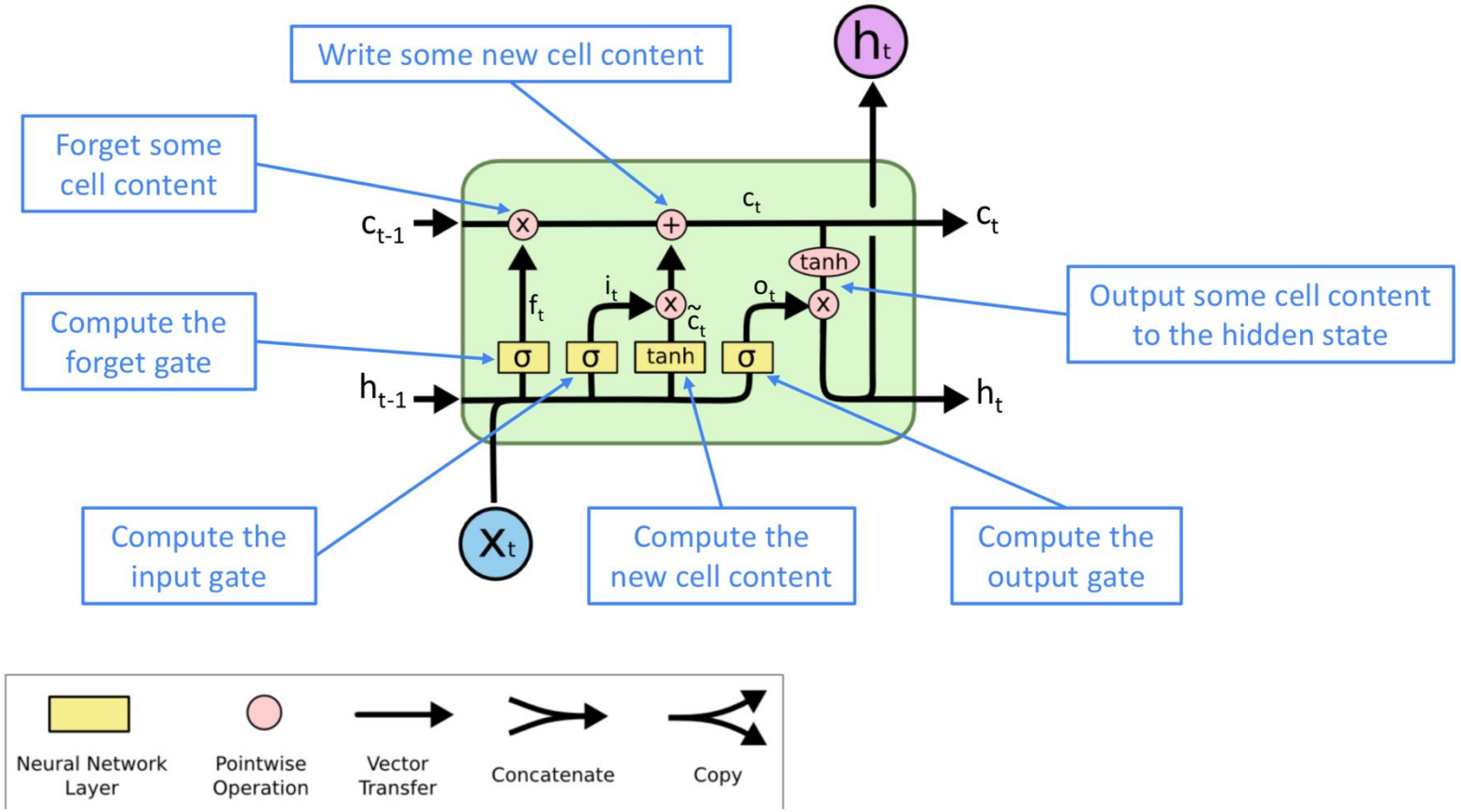
All these are vectors of same length n

Gates are applied using element-wise product

23

Long Short-Term Memory (LSTM)

You can think of the LSTM equations visually like this:



How LSTM Solves the Problem

- The LSTM architecture makes it easier for the RNN to preserve information over many timesteps
- LSTM doesn't *guarantee* that there is no vanishing/exploding gradient, but it does provide an easier way for the model to learn long-distance dependencies

Power of LSTM

- In 2013-2015, LSTMs started achieving state-of-the-art results
  - Successful tasks include: handwriting recognition, speech recognition, machine translation, parsing, image captioning
  - LSTM became the dominant approach
- Now (2019), other approaches (e.g. Transformers) have become more dominant for certain tasks.
  - In WMT 2016, the summary report contains ”RNN” 44 times
  - In WMT 2018, the report contains “RNN” 9 times and “Transformer” 63 times

Gated Recurrent Units (GRU)

- Proposed by Cho et al. in 2014 as a simpler alternative of LSTM
- On each timestep t, we can have input $x^{(t)}$ and hidden state $h^{(t)}$ (no cell state).

Update gate: controls what parts of hidden state are updated vs preserved

$$\boldsymbol{u}^{(t)} = \sigma\left(\boldsymbol{W}_u \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_u \boldsymbol{x}^{(t)} + \boldsymbol{b}_u\right)$$

Reset gate: controls what parts of previous hidden state are used to compute new content

$$\boldsymbol{r}^{(t)} = \sigma\left(\boldsymbol{W}_r \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_r \boldsymbol{x}^{(t)} + \boldsymbol{b}_r\right)$$

New hidden state content: reset gate selects useful parts of prev hidden state. Use this and current input to compute new hidden content.

$$\tilde{\boldsymbol{h}}^{(t)} = \tanh\left(\boldsymbol{W}_h(\boldsymbol{r}^{(t)} \circ \boldsymbol{h}^{(t-1)}) + \boldsymbol{U}_h \boldsymbol{x}^{(t)} + \boldsymbol{b}_h\right)$$

Hidden state: update gate simultaneously controls what is kept from previous hidden state, and what is updated to new hidden state content

$$\boldsymbol{h}^{(t)} = (1 - \boldsymbol{u}^{(t)}) \circ \boldsymbol{h}^{(t-1)} + \boldsymbol{u}^{(t)} \circ \tilde{\boldsymbol{h}}^{(t)}$$

How does this solve vanishing gradient?
Like LSTM, GRU makes it easier to retain info long-term (e.g. by setting update gate to 0)

LSTM vs GRU

- Researchers have proposed many gated RNN variants, but LSTM and GRU are the most widely-used
- The biggest difference is that GRU is quicker to compute and has fewer parameters
- There is no conclusive evidence that one consistently performs better than the other
- LSTM is a good default choice (especially if your data has particularly long dependencies, or you have lots of training data)
- Rule of thumb: start with LSTM, but switch to GRU if you want something more efficient

Are They RNN Problem?

- No, it can be a problem for all neural architectures
  - Due to chain rule/choice of nonlinear functions
  - Lower layers are learned slowly
    - Solution: lots of new architectures with direct connections

For example:

- Residual connections aka "ResNet"
- Also known as skip-connections
- The identity connection preserves information by default
- This makes deep networks much easier to train

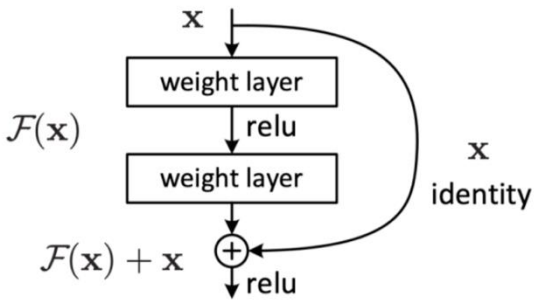


Figure 2. Residual learning: a building block.

Are They RNN Problem?

- No, it can be a problem for all neural architectures
  - Due to chain rule/choice of nonlinear functions
  - Lower layers are learned slowly
    - Solution: lots of new architectures with direct connections

For example:

- Dense connections aka "DenseNet"
- Directly connect everything to everything!

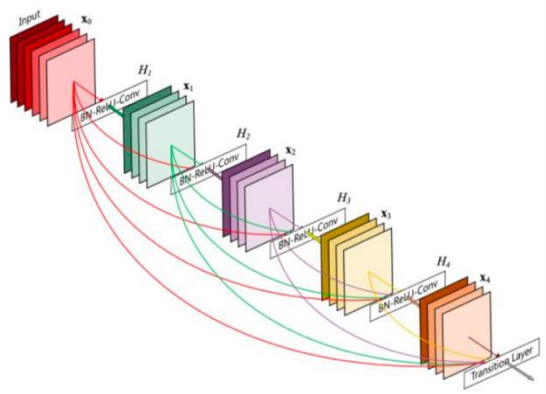


Figure 1: A 5-layer dense block with a growth rate of $k = 4$. Each layer takes all preceding feature-maps as input.

How to Evaluate LMs

- **Perplexity (PPL)**
  - Measures how well a model predicts a sequence.
  - Lower **perplexity** → better model performance.
  - Common for **probabilistic language models** (e.g., RNNs, LSTMs, Transformers).

$$PPL = 2^{(-\frac{1}{N}\sum \log_2 P(w_i))}$$

- **BLEU (Bilingual Evaluation Understudy)**
  - Compares generated text to reference texts using **n-gram precision**.
  - Higher **BLEU** → better match with human-written text.
  - Used in **machine translation, summarization, and text generation**.

How to Evaluate LMs

- **ROUGE (Recall-Oriented Understudy for Gisting Evaluation)**
  - Measures **recall-based overlap** between generated text and reference text.
  - **ROUGE-N**: n-gram recall (e.g., ROUGE-1, ROUGE-2).
  - **ROUGE-L**: Longest common subsequence (LCS) matching.
  - Used for **summarization & text coherence evaluation**.
- **METEOR (Metric for Evaluation of Translation with Explicit ORdering)**
  - Improves BLEU by considering **synonyms & stemming**.
  - Higher correlation with human judgment than BLEU.
  - Used for **machine translation and paraphrase evaluation**.
- **BERTScore**
  - Uses **pretrained embeddings** (e.g., BERT) to compare similarity.
  - Considers **semantic meaning** rather than just word overlap.
  - Used for **advanced NLP evaluation (e.g., dialogue, summarization, translation)**.

How to Evaluate LMs

- **Human Evaluation**
  - **Fluency**: How natural and grammatically correct is the output?
  - **Coherence**: Does the text make sense contextually?
  - **Relevance**: Is the output meaningful to the input prompt?
- **Real-World Testing**
  - **A/B Testing**: Compare model versions in real applications.
  - **User Engagement**: Track user preferences & satisfaction.
  - **Task-Specific Accuracy**: Measure success in applications like chatbots or summarization.